

# Vesuvius Challenge 7位 解法

|          |                      |
|----------|----------------------|
| Rank     | 7                    |
| Team     | DECEM                |
| Author   | DECEM                |
| Topic ID | 679251               |
| Version  | Japanese Translation |

Source: <https://www.kaggle.com/competitions/vesuvius-challenge-surface-detection/discussion/679251>

Retrieved with Kaggle CLI on 2026-07-07.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

この素晴らしいコンペを開催してくれた主催者に感謝する。volumetric topologyを深く調べ、多くを学び楽しめた。受賞者・参加者の皆さん、おめでとう。以下が私の手法の概要である。

## 概要

---

7位、Public LB 0.618 / Private LB 0.591。多くのチームと同様、nnU-Netが主役だった。2つのnnU-Netモデルのensemble + TTA + post-processingという単純なpipelineである。

## モデル訓練

---

両モデルとも nnUNetPlannerResEncM、3d\_lowres 構成のnnUNetを使用した。

- Patch size: **128<sup>3</sup>**
- Epochs: **2000**
- Optimizer: **SGD** (nnUNet default: lr=0.01, momentum=0.99, nesterov=True, wd=3e-5)
- Loss: **CE Loss + Dice Loss + Skeleton Recall Loss** (すべてweight 1.0)

2モデルの主な違い：

**Model A — 前処理済み訓練label (重要な発見)**：organizerのGitHub repoにtest set専用の処理pipelineがあることに気づき、訓練labelにも**まったく同じ処理**を適用した。これはおそらく最大の改善だった。生の訓練labelとtest時処理にはずれがあり、汎化を損なっていた。train/testのlabel処理を揃えると明確で一貫した改善が得られた。

**Model B — 生の訓練label**：label前処理なしの標準訓練で、ensembleに多様性を与えた。

## Ensembleと推論

---

- Model A + Model Bの単純ensemble
- **TTA** (nnUNet組み込みのmirror test-time augmentation)

特別な工夫はなく、nnUNetに任せた。

## Post-processing

---

nnU-Netのbinary予測へ4段階の処理を行い、topology、surface精度、instance一貫性を改善した。

**Ridge Detection (Hessian-based Frangi Filter)**：3D Frangi sheetness filterをbinarize済みvolumeへ直接適用する。各voxelで2階Gaussian derivativeからHessianを計算し、固有値を $|\lambda_1| \leq |\lambda_2| \leq |\lambda_3|$ に並べてsheetness responseを求める。平面・sheet状構造は $|\lambda_1| \approx 0, |\lambda_2| \approx 0, |\lambda_3| \gg 0$ となり、パピルスsurface形状に対応する。responseはbackground抑制、planarity、blob除外を組み合わせ、bright-on-dark構造 ( $\lambda_3 < 0$ ) のみを残す。threshold後のbinary maskは連続surfaceを強め、孤立noiseと非平面false positiveを抑える。

**Coherence Enhancing Diffusion (CED)** : ridge detection後、Frangi処理済みvolumeへslice単位の2D CEDを適用し、surface方向の連続性を高める。CEDは $\partial u / \partial t = \text{div}(D(J\rho) \cdot \nabla u)$ で支配されるPDE型anisotropic diffusionで、diffusion tensor Dはstructure tensor  $J\rho$ から得る。Pavel Holoborodko derivative kernelとGaussian smoothingでstructure tensorを求め、surface接線方向へ強く拡散し、垂直方向のedgeを保つ。PyTorchでGPU化し、再binarizeした。edgeの鋭さを保ちながら小穴を埋め、分断segmentを再接続し、境界noiseを平滑化する。

**Anisotropic Morphological Closing** : anisotropic ellipsoid structuring elementで3D binary closing (dilation後erosion) を行う。scan artifactによるdepth方向の小gapを考慮してZ-radiusをXY-radiusより大きくした。隣接layerをXY面で過剰接続せず、CED後に残る小穴や微細な断裂を埋める。

**Dust Removal** : voxel数threshold未満のsmall connected componentを除去し、意味のあるpapyrus構造にならないnoise fragmentやfalse positiveを消す。

## Pipeline概要

```
Binary Prediction (nnU-Net) + TTA
  → Frangi Sheetness Filter (3D)
  → CED Anisotropic Diffusion (2D slice-wise, GPU)
  → Anisotropic Closing
  → Dust Removal
  → Final Prediction
```

## Muon Optimizerについて

SGDの代わりに**Muon optimizer**も実験し、Muon訓練モデルはPublic/Private LBの両方で一貫してSGDを上回った。しかし全最終モデルを訓練するcomputeがなく、提出ensembleはSGDモデルである。SGDはMuonより約**1.25x高速**にすぎず、大きなtradeoffではない。

最適化版Muon実装**FastMuon**を公開した。Triton加速のdrop-in Turbo-Muonで、hardwareがあれば試す価値がある。

## 効かなかった／試せなかったこと

- 強いdata augmentation : 強度・確率を上げるほど悪化し、nnUNet defaultより低い確率にした
- cLDice Loss : CE + Dice + Skeleton Recallより改善せず
- UNetへのSE module追加 : 有意な改善なし、VRAMだけ増加
- pseudo label : unlabeled data予測のnoiseが大きく、semi-supervised trainingは悪化
- 強いtopological post-processing (PCA hole filling、Betti matchingなど)

DECEM

## 補足Q&A

**Giorgio Angelotti**: Muon optimizerについて、もう少し結果を共有してもらえますか？

**DECEM**: 同じ構成 (nnUNetPlannerResEncM、3d\_lowres、128<sup>3</sup> patch、single fold、同じpost-processing) で1000 epochs訓練し、optimizerだけ交換した。

| Optimizer  | Implementation / key params                                                                                                       | Private LB |
|------------|-----------------------------------------------------------------------------------------------------------------------------------|------------|
| SGD        | <code>torch.optim.SGD; lr=0.01, momentum=0.99, nesterov=True, wd=3e-5</code>                                                      | 0.590      |
| AdamW      | <code>torch.optim.AdamW; lr=3e-4, betas=(0.9,0.999), eps=1e-8, wd=3e-5</code>                                                     | 0.594      |
| Muon+AdamW | <code>SingleDeviceMuonWithAuxAdam; Muon lr=3e-4, momentum=0.95, turbo; AdamW lr=3e-4, betas=(0.9,0.95), eps=1e-10, wd=0.01</code> | 0.595      |

FastMuonのSingleDeviceMuonWithAuxAdamを使い、conv/linear weight matrix (ndim>=2、embed/head/seg\_layersを除く)へTurbo mode (AOL preconditioning + Newton-Schulz 4 iterations)、bias・normalization・segmentation headへ組み込みAdamWを適用した。全てPolyLR scheduling。SGDはMuonより約1.25x速いだけで、Muonのhyperparameter (推奨されるlr=0.02など)はcompute不足で十分探索できていない。適切にtuningすれば差は広がる可能性がある。